Lesson 2 — Python Basics

READ ME FIRST (theory). Then open the practice notebook.

Contents

Lesson 2 — Python Basics

This is the **theory**. Read it slowly, top to bottom. You do **not** write code here. When you finish reading, open `lesson_2_practice.ipynb`, run every cell, and do the assignment.

How to read this lesson: for each idea you will see (1) what it is, (2) a small example, (3) **why** we use it, and (4) a **common mistake** to avoid.

Key words in this lesson

Word	Simple meaning
variable	a box with a name that holds a value
string	text (letters), always inside quotes
integer	a whole number (no decimal)
float	a number with a decimal point
boolean	True or False
list	many values in order
dictionary	values with labels (key → value)
function	a small machine: input → work → output

What is Python?

Python is a language we use to talk to the computer. We write simple instructions. The computer reads them **one line at a time, top to bottom**, and does what they say.

In Colab, we write code inside a **cell**. We press **Shift + Enter** to run that cell. The result appears under it.

---

1. Variables — boxes that hold a value

A **variable** is a box with a name. We put a value inside, using the `=` sign.

```
name = "Ali"
age = 25
```

- Text goes inside quotes: `"Ali"`.
- Numbers have no quotes: `25`.

You can use the box again later, and you can change what is inside:

```
age = 25
age = age + 1      # now age is 26
```

Why: so we can save a value and use it again later, just by its name.

Common mistake: writing `name = Ali` without quotes. Python will look for a box called `Ali` and crash. Text always needs quotes.

2. Data types — the kind of value

Every value has a **type**:

Type	Example	Meaning
string (`str`)	`"Tashkent"`	text
integer (`int`)	`42`	whole number
float	`19.99`	number with a decimal
boolean (`bool`)	`True` / `False`	yes / no

You can check the type with `type(x)`.

Why it matters: the type decides what you can do. You can do math with numbers, not with text.

---

3. Text is not a number

`"30"` (with quotes) is **text**. `30` (no quotes) is a **number**.

You cannot do math with text. To fix this:

- `int("30")` → the number `30`
- `str(30)` → the text `"30"`

```
age_text = "30"
print(int(age_text) + 5)     # 35
```

Why: data very often arrives as text (from files, websites). We must change it into numbers before we can add or multiply.

Common mistake: `"30" + 5` → error. You must change the text to a number first.

---

4. Math and comparison

Python does normal math: `+` `-` `*` `/`. Two special ones:

- `//` = divide and drop the decimal (`7 // 2` → `3`)
- `%` = the remainder (`7 % 2` → `1`)

Comparison asks a yes/no question and gives `True` or `False`:

Symbol	Meaning
`==`	equal to
`!=`	not equal to
`> <`	bigger / smaller
`>= <=`	bigger-or-equal / smaller-or-equal

Common mistake: using `=` (put a value in a box) when you mean `==` (check if equal). They are different!

---

5. f-strings — putting values inside text

An **f-string** lets you put a variable inside a sentence. Put `f` before the quotes and the variable in `{ }`.

```
name = "Ali"
age = 25
print(f"{name} is {age} years old")    # Ali is 25 years old
```

You can also round numbers: `f"{price:.2f}"` shows 2 decimals.

Why: to make clear, readable messages for the user.

---

6. if / elif / else — making a choice

The computer checks a question. If the answer is **True**, it does one block. If not, it checks the next, or does `else`.

```
if score >= 90:
    print("excellent")
elif score >= 60:
    print("pass")
else:
    print("fail")
```

You can join questions with `and`, `or`, `not`:

- `and` → both must be True
- `or` → at least one is True
- `not` → the opposite

Why: real decisions depend on conditions.

Common mistake: forgetting the `:` at the end of the `if` line, or forgetting to indent the line under it. Python uses spaces (indentation) to know what belongs inside.

---

7. for loop — repeat for each item

A **loop** repeats the same action for every item in a list.

```
total = 0
for n in [5, 10, 15, 20]:
    total = total + n
print(total)        # 50
```

`range(5)` gives the numbers `0, 1, 2, 3, 4` — useful to repeat a fixed number of times.

Two helpers inside a loop:

- `break` → stop the loop now
- `continue` → skip this item, go to the next

Why: we do not want to write the same line 100 times.

Common mistake: forgetting the indentation. The lines inside the loop must be indented.

---

8. list — many items in order

A **list** holds many values together:

```
prices = [10, 20, 30, 40, 50]
```

We pick items by their **position (index)**. The first position is `0`, not 1.

Code	Result	Meaning
`prices[0]`	`10`	first item
`prices[-1]`	`50`	last item
`prices[1:3]`	`[20, 30]`	position 1 and 2 (stop **before** 3)
`len(prices)`	`5`	how many items
`prices.append(60)`	—	add `60` to the end

Why: data is usually many values together (all the prices, all the customers).

Common mistake: thinking the first item is position 1. In Python it is position **0**.

---

9. dictionary — pairs of key and value

A **dictionary** stores information with labels:

```
customer = {"name": "Ali", "city": "Tashkent", "age": 30}
```

We pick a value by its **key name**:

Code	Result
`customer["name"]`	`"Ali"`
`customer.get("plan", "basic")`	`"basic"` (no "plan" key → safe default)
`customer.keys()`	all the keys
`customer.values()`	all the values

Why: it keeps information with clear labels, easy to find by name.

Common mistake: `customer["plan"]` when there is no "plan" key → crash. Use `.get("plan", "basic")` to stay safe.

---

10. function — a small machine you reuse

A **function** takes an input, does some work, and gives back an output with `return`. An input can have a **default value**.

```
def price_with_tax(x, tax=10):
    return x + x * tax / 100

price_with_tax(200)       # 220
price_with_tax(200, 5)    # 210
```

Why: write the code once, use it many times. Cleaner and fewer mistakes.

Common mistake: forgetting `return`. Without it, the function gives back `None` (nothing).

Quick summary

- **Variable** = a named box. Text needs quotes, numbers do not.
- **Change type** with `int()` and `str()`.
- **if / elif / else** = choices. **for** = repeat.
- **list** = ordered values (start at index 0). **dictionary** = labeled values.
- **function** = reusable machine, ends with `return`.

Now do this

1. Open `lesson_2_practice.ipynb`.
2. Run every cell (Shift + Enter) and watch the result.
3. Try changing a value and run again — see what happens.
4. Do the **assignment** at the bottom.

   Read the theory first, then see it work. Reading + doing = understanding.